2.1. Файлы и файловая система

Необходимо знать:

- понятие файла;

Файл — это именованная область данных на накопителе (SSD/HDD), представляющая собой упорядоченную последовательность байтов.

- понятие каталога;

Каталог (папка / директория) — это специальный служебный файл, который содержит список других файлов и каталогов, связывая их имена с системными индексами файловой системы (например, в NTFS).

- абсолютные и относительные пути;

Абсолютный путь — полный путь от буквы диска (корня файловой системы)

Относительный путь — путь относительно текущей рабочей папки программы

- файловый объект Windows;

Файловый объект (File Object) — это системная структура внутри памяти ядра Windows (FILE_OBJECT). В ней ОС хранит всё: где файл на диске, сколько байт открыто, текущую позицию чтения/записи, права доступа.

- файловый дескриптор HANDLE;

Дескриптор (HANDLE) — это «номерок» в руках твоей программы, через который ты командуешь файловым объектом ядра.

Особое значение при ошибке:

Если CreateProcess или CreateThread при ошибке возвращали NULL, то файловая функция CreateFile при ошибке возвращает INVALID_HANDLE_VALUE (число -1). Это частая ловушка на защите!

- права доступа к файлу;

При открытии файла ты обязана сказать Windows, что ты собираешься делать:

- GENERIC_READ — только читать;
- GENERIC_WRITE — только писать;
- GENERIC_READ | GENERIC_WRITE — и читать, и писать.

- режимы совместного использования файла;

Что делать, если твой файл одновременно захочет открыть другая программа (например, «Блокнот»)?

- 0 — монопольный доступ. Файл заблокирован намертво. Никто в системе не может даже прочитать его, пока ты не вызовешь CloseHandle.
- FILE_SHARE_READ — разрешить другим параллельно читать файл.
- FILE_SHARE_WRITE — разрешить другим параллельно писать в файл.

- создание и открытие файла;

- файловый указатель;

Это внутренний маркер в файловом объекте (позиция «курсора» в файле от 0 до размера файла):

Открыли пустой файл → курсор на байте 0.

Записали 100 байт → курсор автоматически сдвинулся на байт 100. Следующая запись начнется со 100-го байта.

Смещать курсор вручную можно функцией SetFilePointerEx().

- размер файла;

- завершение работы с файловым объектом.

Основные функции Windows API:

CreateFile()

CreateFile() — открывает существующий файл или создает новый (в зависимости от флагов: CREATE_ALWAYS, OPEN_EXISTING и т.д.).

CloseHandle()

CloseHandle() — закрывает файловый дескриптор. Когда все дескрипторы файла закрыты, Windows сбрасывает остатки данных из кэша на диск и удаляет файловый объект ядра.

GetFileSizeEx()

GetFileSizeEx() — узнает точный размер файла в байтах. Записывает результат в 64-битное число LARGE_INTEGER, что позволяет узнавать размер файлов больше 4 ГБ.

SetFilePointerEx()

SetFilePointerEx() — перемещает файловый курсор на произвольную позицию (например, прыгнуть сразу на 500-й мегабайт файла).

SetEndOfFile()

SetEndOfFile() — устанавливает конец файла на текущую позицию курсора. С помощью этой функции можно мгновенно обрезать файл или увеличить его до гигантского размера.

FlushFileBuffers()

FlushFileBuffers() — принудительно заставляет Windows немедленно сбросить все несохраненные данные из оперативной памяти (файлового кэша) прямо на физический диск (защита от потери данных при выключении питания).

2.2. Синхронный ввод-вывод

При синхронной операции программа инициирует операцию чтения или записи и ожидает ее завершения.

Основные функции:

ReadFile()

WriteFile()

1. Поток программы засыпает (блокируется).
2. Операционная система Windows отправляет команду контроллеру диска: «Прочитай мне этот сектор».
3. Программа ничего не делает, пока диск физически не считает байты и не скопирует их в память.
4. Только после этого функция возвращает управление, и программа идет к следующей строчке.

Необходимо понимать:

- последовательное чтение;

Если читать файл размером 100 МБ по 1 байтику — программа сделает 100000000 системных вызовов к диску. Компьютер просто зависнет от колоссальных накладных расходов.

- чтение блоками;

Если читать файл блоками (буфером), например по 64 КБ или 1 МБ — программе потребуется всего 100 обращений к ядру. Скорость вырастет в сотни раз! Именно поэтому в генераторе файла мы писали буфером по 1 МБ.

- запись блоками;

- позиционирование внутри файла;

- влияние размера блока на производительность.

2.3. Асинхронный ввод-вывод

Асинхронный ввод-вывод позволяет инициировать операцию и продолжить выполнение программы до ее завершения.

Файл открывается с флагом:

FILE_FLAG_OVERLAPPED.

Для каждой независимой операции используется структура:

OVERLAPPED.

1. Файл открывается со специальным флагом: FILE_FLAG_OVERLAPPED.
2. Мы вызываем ReadFile() или WriteFile().
3. Функция НЕ ЖДЕТ диска, она возвращает управление мгновенно!
4. При этом функция возвращает FALSE, а системная ошибка GetLastError() равна специальному коду: ERROR_IO_PENDING (что означает: «Операция успешно начата и прямо сейчас выполняется в фоне контроллером диска»).
5. Наша программа может в это время рисовать интерфейс, считать матрицы или делать что угодно.

Что такое структура OVERLAPPED?

Поскольку мы не стоим на месте, у асинхронной операции нет понятия «текущего курсора файла».

Поэтому для каждой операции мы обязаны передать отдельную структуру OVERLAPPED. В ней хранятся:

- Смещение в файле (с какого байта читать).
- Системное событие hEvent (Event) — тот самый «колокольчик», который Windows переведет в сигнальное состояние, когда диск реально закончит читать байты.

Контроль завершения (GetOverlappedResult)

Когда программе наконец понадобились прочитанные данные, она вызывает:

- WaitForSingleObject(overlapped.hEvent, INFINITE) — подождать, пока диск закончит фоновое чтение.
- GetOverlappedResult() — узнать, сколько байт в итоге реально прочиталось с диска.

Основные функции и объекты:

ReadFile()

WriteFile()

GetOverlappedResult()

WaitForSingleObject()

WaitForMultipleObjects()

CreateEvent()

ResetEvent()

Необходимо понимать:

- начало асинхронной операции;

- значение ERROR_IO_PENDING;

- контроль завершения операции;

- одновременное выполнение нескольких операций;

- необходимость использования отдельной структуры OVERLAPPED для каждой незавершенной операции.

2.4. Отображение файла в память

Отображение файла позволяет работать с содержимым файла как с участком виртуальной памяти процесса.

Основные функции:

CreateFileMapping()

MapViewOfFile()

UnmapViewOfFile()

FlushViewOfFile()

Необходимо понимать:

Memory Mapping – это механизм ядра Windows, связывающий страницы файла на накопителе с диапазоном виртуальных адресов процесса. Вместо функций ReadFile/WriteFile мы работаем с файлом через обычные указатели в оперативной памяти (char* ptr).

- объект отображения файла;

Представь, что у тебя есть большая книга (файл на диске), лежащая в закрытом архиве.

Файловый дескриптор (hFile от CreateFile) — это просто пропуск в архив к этой книге.

Объект отображения (hMapping от CreateFileMapping) — это специальное системное соглашение (договор) с операционной системой Windows о том, как именно эту книгу разрешено читать через память:

- В каком режиме (только чтение PAGE_READONLY или с правом записи PAGE_READWRITE).
- Какой максимальный объем книги вообще планируется проецировать.

- представление файла;

Представление (View) — это конкретный диапазон виртуальной памяти процесса, в который операционная система прямо сейчас спроецировала байты файла.

- смещение отображения;

Это точка старта в файле, с которой мы начинаем проецировать представление (номер первого байта окна).

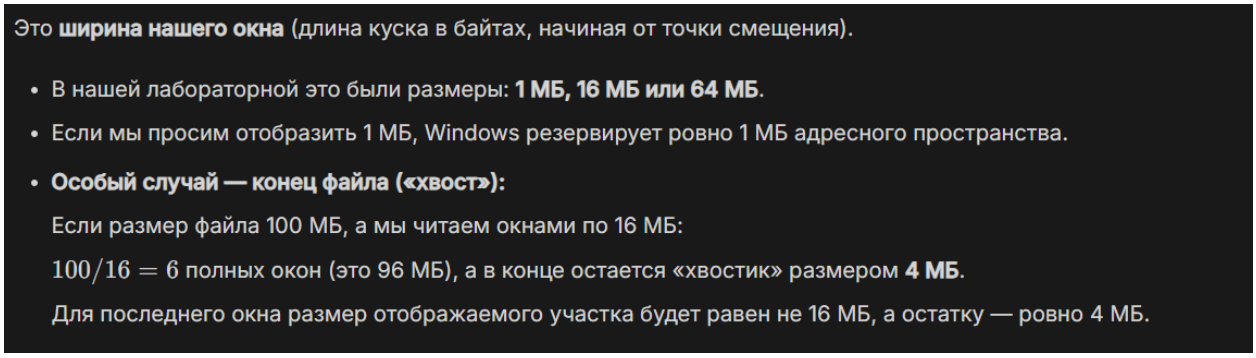
- На 1-м шаге смещение = 0 (начало файла).
- На 2-м шаге смещение = 1 МБ (байт № 1 048 576).
- На 3-м шаге смещение = 2 МБ и так далее.

Главное системное правило Windows (Выравнивание):

Смещение в файле не может быть произвольным числом! Оно обязано быть кратно гранулярности распределения памяти операционной системы (Allocation Granularity), которая в Windows всегда равна 64 КБ (65 536 байт).

Если передать смещение, не кратное 64 КБ (например, 100-й байт), функция MapViewOfFile упадет с системной ошибкой ERROR_MAPPED_ALIGNMENT.

- размер отображаемого участка;



- отличие отображения файла от обычного ReadFile() / WriteFile();

Главные преимущества перед ReadFile/WriteFile:

Нулевое копирование (Zero-Copy): данные не копируются из кэша ядра в буфер пользователя. Процесс обращается напрямую к кэш-страницам памяти.

Случайный доступ: можно мгновенно прочитать любой байт по индексу ptr[12345], не вызывая медленный SetFilePointerEx.

Экономия ОЗУ (Lazy Loading / Ленивая загрузка): Windows загружает данные с диска в память не сразу, а только в тот момент, когда процессор реально пытается прочитать байт по адресу (механизм страничного прерывания — Page Fault).

- возможность отображения частей большого файла.

Цепочка вызовов (Твой вариант):

- CreateFileMapping() — создает объект ядра «проекция файла».
- MapViewOfFile() — проецирует кусок файла (окно) в память, возвращает указатель.
- UnmapViewOfFile() — отключает окно, освобождая адреса.
- FlushViewOfFile() — принудительно сохраняет измененные страницы памяти обратно в файл на диске.

2.5. Мультиплексирование операций ввода-вывода

Под мультиплексированием в данной лабораторной работе понимается возможность одной программы контролировать несколько одновременно выполняемых операций ввода-вывода.

Для асинхронных файловых операций рекомендуется использовать:

WaitForMultipleObjects()

с событиями, связанными со структурами OVERLAPPED.

Для сокетов допускается использование:

select().

Следует учитывать, что в Windows функция select() относится прежде всего к Winsock и не предназначена для обычного ожидания файловых HANDLE.

1. Что такое файловый дескриптор HANDLE?

Дескриптор (HANDLE) — это программный идентификатор (индекс в системной таблице дескрипторов процесса), через который приложение получает контролируемый доступ к системному файловому объекту (FILE_OBJECT), расположенному в защищенной памяти ядра Windows

2. Для чего используется CreateFile()?

Функция CreateFile() является универсальным швейцарским ножом Windows API и используется для создания новых файлов, а также открытия уже существующих файлов, каталогов, физических дисков и последовательных портов

3. Какие основные параметры принимает CreateFile()?

    1. lpFileName — имя (или путь) файла на диске.
    2. dwDesiredAccess — права доступа (что мы хотим делать: читать GENERIC_READ или писать GENERIC_WRITE).
    3. dwShareMode — режим совместного доступа (разрешаем ли другим процессам параллельно читать/писать в этот файл).
    4. dwCreationDisposition — действие над файлом (создать новый, открыть старый, перезаписать).

4. Для чего используются параметры dwDesiredAccess и dwShareMode?

- `dwDesiredAccess` **(Desired = Желаемый доступ):**
  Это то, **ЧТО ТЫ САМА** хочешь делать с файлом прямо сейчас:
  - `GENERIC_READ` — «я буду только читать»;
  - `GENERIC_WRITE` — «я буду писать»;
  - `GENERIC_READ | GENERIC_WRITE` — «и читать, и писать».
- `dwShareMode` **(Share = Делиться / Совместный доступ):**
  Это то, **ЧТО ТЫ РАЗРЕШАЕШЬ ДЕЛАТЬ ДРУГИМ** программам, пока файл у тебя открыт:
  - `0` — монопольный режим (никому нельзя даже заглянуть в файл, пока ты его не закроешь);
  - `FILE_SHARE_READ` — другие программы могут параллельно читать этот же файл.

5. Чем CREATE_ALWAYS отличается от OPEN_EXISTING?

- `CREATE_ALWAYS` («Создать всегда»):
  Если файла **нет** → Windows создаст новый пустой файл.
  Если файл **уже есть** → Windows **сотрет (очистит) старый файл** до нуля байт и создаст на его месте чистый пустой файл (перезапись).
- `OPEN_EXISTING` («Открыть существующий»):
  Если файл **есть** → Windows успешно откроет его, сохранив все данные внутри.
  Если файла **нет** → Windows **НЕ станет его создавать**, а выдаст ошибку `ERROR_FILE_NOT_FOUND` (файл не найден).

6. Как определить размер файла?

В Windows API для этого используется системная функция **GetFileSizeEx().**

Она принимает дескриптор открытого файла hFile и записывает его точный размер в 64-битную структуру LARGE_INTEGER (что позволяет правильно определять размер даже гигантских файлов больше 4 ГБ)

7. Для чего используется SetFilePointerEx()?

«Она используется для **произвольного позиционирования файлового указателя** (курсора чтения/записи) внутри файла. С её помощью можно переместить указатель на любое количество байтов:

- от начала файла (`FILE_BEGIN`);
- от текущей позиции (`FILE_CURRENT`);
- от конца файла назад (`FILE_END`)».

8. Чем синхронный ввод-вывод отличается от асинхронного?

- При синхронном — вызывающий поток **блокируется (усыпляется ядром)** до окончания работы диска.
- При асинхронном — функция `ReadFile`/`WriteFile` **возвращает управление мгновенно**, а передача данных идет в фоне через контроллер диска (DMA), пока поток занят другими вычислениями.

9. Для чего используется FILE_FLAG_OVERLAPPED?

«Это специальный флаг, который передается в функцию `CreateFile()`. Он сообщает операционной системе: **«Этот файл открывается в АСИНХРОННОМ режиме!»**
Если этот флаг НЕ указать, Windows будет выполнять все операции с файлом строго синхронно (с блокировкой потока), даже если мы попытаемся передать структуру `OVERLAPPED`».

10. Что представляет собой структура OVERLAPPED?

Зачем она вообще нужна:
Когда мы читаем файл асинхронно, у нас **нет понятия текущего курсора**, потому что одновременно может идти 5 разных операций чтения в разные части файла!

Что внутри структуры (ответ для преподавателя):
«Структура `OVERLAPPED` описывает параметры конкретной асинхронной операции и содержит:

1. **Смещение в файле** (`Offset` и `OffsetHigh`) — с какого именно байта файла производить чтение или запись;
2. **Дескриптор события** `hEvent` — системное событие (Event), которое Windows автоматически переведет в сигнальное состояние («позвонит в звоночек»), когда операция на диске физически завершится;
3. Внутренние системные поля ядра (`Internal`, `InternalHigh`) для кода ошибки и количества переданных байт».

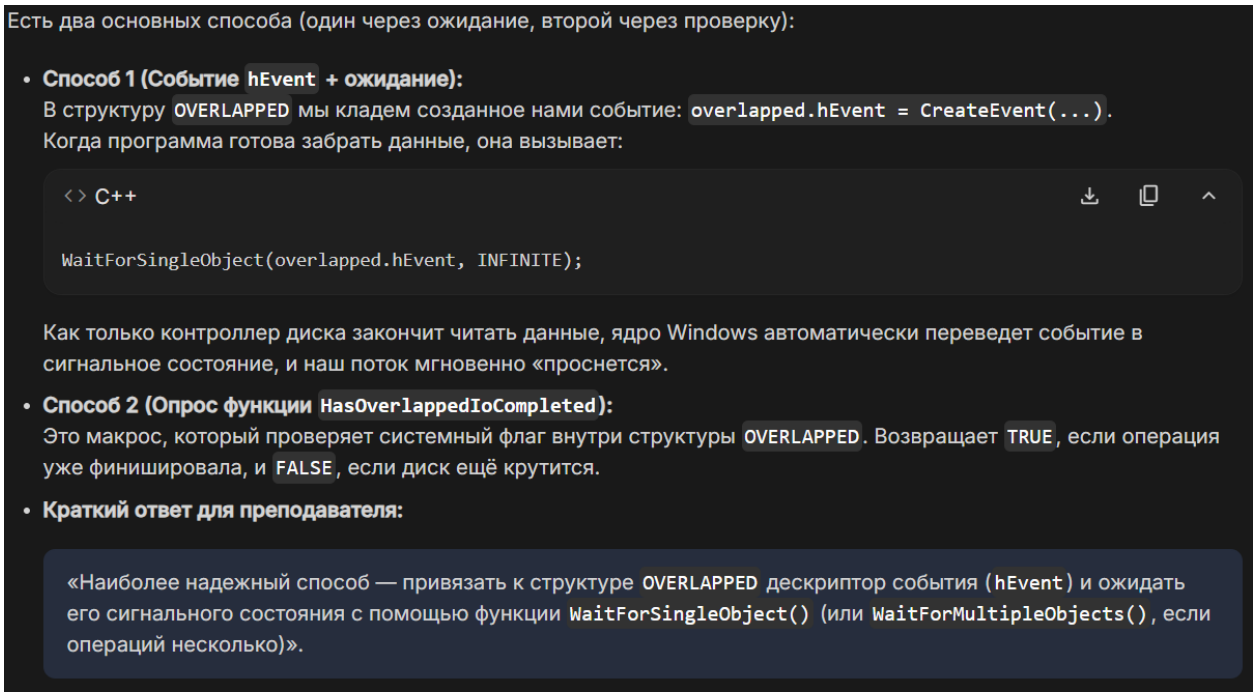
11. Что означает ошибка ERROR_IO_PENDING?

«Когда мы запускаем асинхронный вызов `ReadFile()` или `WriteFile()`, функция возвращает `FALSE`, а `GetLastError()` возвращает `ERROR_IO_PENDING`.
Это означает: **«Операция ввода-вывода успешно поставлена в очередь и прямо сейчас выполняется в фоне»**. Это штатное поведение асинхронного ввода-вывода Windows».

12. Почему нельзя использовать одну структуру OVERLAPPED одновременно для нескольких незавершенных операций?

Потому что операционная система Windows напрямую использует поля структуры OVERLAPPED для отслеживания состояния, смещения и записи результатов конкретной фоновой операции. Если передать одну и ту же структуру в две операции одновременно, возникнет перезапись системных полей ядра и непредсказуемое поведение (повреждение данных)

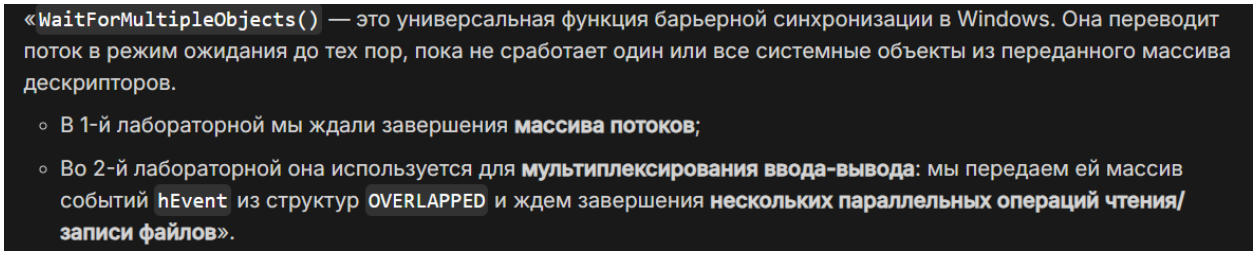
13. Как определить завершение асинхронной операции?



14. Для чего используется GetOverlappedResult()?

Функция GetOverlappedResult() используется для получения итоговых результатов асинхронной операции: она возвращает фактическое количество прочитанных или записанных байтов, а также сообщает статус выполнения — завершилась ли операция успешно или произошла ошибка

15. Для чего используется WaitForMultipleObjects()?



16. Что такое отображение файла в память?

«**Отображение файла в память** — это механизм операционной системы, который связывает содержимое файла на диске напрямую с **виртуальным адресным пространством процесса**.
Вместо чтения через буферы программа получает обычный указатель в памяти (`char* ptr`) и работает с файлом как с обычным массивом в оперативной памяти».

17. Для чего используется CreateFileMapping()?

«Она создает в ядре операционной системы специальный системный объект — **Объект отображения файла (File Mapping Object)**. Он задает правила: права доступа к памяти (`PAGE_READONLY` или `PAGE_READWRITE`) и максимальный размер проецируемой области».

18. Для чего используется MapViewOfFile()?

«Функция `MapViewOfFile()` выполняет **непосредственное проецирование** выбранного участка файла в виртуальную память процесса.
Она принимает смещение в файле и размер окна, резервирует виртуальные адреса и **возвращает прямой указатель (`void*`) на байты файла в памяти**. Именно через этот указатель мы читаем данные!»

19. Чем отображение файла отличается от ReadFile()?

1. **Принцип нулевого копирования (Zero-Copy):**
   При `ReadFile()` ядро ОС сначала читает файл в свой системный кэш, а затем **копирует** байты во второй раз — в буфер твоей программы (тратится память и время CPU). При отображении данных копирования нет: программа обращается к страницам файла напрямую!
2. **Удобство произвольного доступа:**
   При `ReadFile()` нужно двигать курсор через `SetFilePointerEx`. При отображении ты просто обращаешься к любому байту по индексу: `ptr[12345]`.

20. Почему отображение файла не означает, что весь файл сразу загружается в физическую память?

«Windows использует механизм **«ленивой загрузки» (Lazy Loading)** на основе виртуальной памяти:

1. При вызове `MapViewOfFile()` резервируются только **виртуальные адреса** (номера страниц), но сам файл с диска не читается, физическая память (ОЗУ) не расходуется!
2. Физическая страница файла подгружается с диска в ОЗУ **только в тот микросекундный момент, когда процессор реально пытается прочитать байт по этому адресу**.
3. Это аппаратное событие процессора называется **Страничным прерыванием (Page Fault)**. Если мы не обращаемся к странице — она никогда не будет загружена в оперативку».

21. Для чего используется FlushViewOfFile()?

- **Смысл функции:**
  Когда мы отображаем файл с правом записи (`PAGE_READWRITE`) и меняем байты через указатель `ptr[i] = 42;`, изменения происходят **только в оперативной памяти (в кэше страниц)**. На жесткий диск они сбрасываются не сразу, а когда Windows сочтет нужным.
- **Ответ для преподавателя:**
  «Функция `FlushViewOfFile()` используется для **принудительного немедленного сброса (записи) измененных страниц памяти из оперативной памяти обратно на физический накопитель (диск)**.
  Это аналог функции `FlushFileBuffers()`, гарантирующий, что записанные в память данные не потеряются при аварийном завершении питания».

22. Что происходит при обращении к странице отображенного файла, которая еще не находится в физической памяти?

- **Что происходит внутри «железа» компьютера:**
  1. Твоя программа пытается прочитать байт по адресу (например, `char c = ptr[0];`).
  2. Блок управления памятью процессора (MMU) видит: в таблице страниц стоит флаг *«страницы нет в ОЗУ»*.
  3. Процессор ставит выполнение программы на паузу и генерирует специальное аппаратное прерывание — **Страничный сбой (Page Fault)**.
  4. Управление мгновенно перехватывает ядро Windows (диспетчер виртуальной памяти).
  5. Ядро Windows само считывает нужный кусок файла с жесткого диска в свободную ячейку физической оперативной памяти (RAM).
  6. Ядро обновляет таблицу страниц и возобновляет работу твоей программы.
     Для твоей программы всё выглядит так, будто байт просто мгновенно прочитался из памяти!

23. Почему размер блока ввода-вывода влияет на производительность?

- **Почему так:**
  Каждое обращение к диску (хоть через `ReadFile`, хоть через `MapViewOfFile`) требует **накладных расходов операционной системы**: переключение в режим ядра (Kernel Mode), поиск секторов на диске, создание системных запросов (IRP).
  - **Если размер блока крошечный (например, 4 КБ):** программа сделает миллионы системных вызовов. Процессор потратит 90% времени на системную бюрократию, а не на чтение.
  - **Если размер блока оптимальный (1 МБ — 16 МБ):** накладные расходы снижаются до минимума, а диск считывает данные непрерывным потоком на максимальной физической скорости.
  - *Слишком большой блок (например, 1 ГБ)* тоже плох — он может забить всю оперативную память.

24. Почему асинхронный ввод-вывод не всегда быстрее синхронного?

1. **Накладные расходы:** Асинхронный ввод-вывод сложнее: ядру нужно выделить структуры, зарегистрировать событие `hEvent`, потом проверить завершение через `GetOverlappedResult`. Если файл лежит в быстром кэше памяти или на супербыстром NVMe SSD, синхронный вызов отработает за микросекунду, а асинхронная бюрократия займет больше времени!
2. **Один физический диск:** Если отправлять много асинхронных запросов к обычному механическому HDD, его считывающая головка начнет судорожно метаться по пластинам (Thrashing), и скорость упадет в разы.

25. Почему повторный запуск программы может работать быстрее первого?

- **Настоящая причина: ФАЙЛОВЫЙ КЭШ ОПЕРАЦИОННОЙ СИСТЕМЫ (System Cache)!**
  - При **первом запуске** (холодный старт) данные физически читаются с медленного диска (SSD/HDD).
  - Windows очень умная: прочитав файл один раз, она **оставляет его копию в свободной оперативной памяти (в системном кэше)**.
  - При **повторном запуске** (горячий старт) Windows видит: *«О, этот файл уже лежит в оперативной памяти!»* и отдает его со скоростью ОЗУ (десятки гигабайт в секунду), вообще не обращаясь к диску!

26. Что такое файловое кэширование?

- **Определение:** Это механизм операционной системы, при котором часто используемые или недавно прочитанные/записанные блоки файлов временно сохраняются в **быстрой оперативной памяти (ОЗУ)**.
  Это позволяет приложениям читать данные из памяти, минимизируя медленные физические обращения к диску.

27. В чем преимущество записи фиксированной длины для файла прямого доступа?

Математика прямого доступа:

Если каждая запись в файле весит ровно, например, 128 байт:

Чтобы мгновенно найти запись под номером N, не нужно читать весь файл от начала. Мы можем вычислить её точный адрес за одну операцию умножения:

$$\text{Смещение} = N \times 128$$

И сразу прыгнуть туда через `SetFilePointerEx` или смещение отображения за $O(1)$!

28. Как можно организовать удаление записей без немедленного физического удаления?

В начало каждой записи добавляют 1 служебный байтик (флаг):

- `0` — запись активна (жива);
- `1` — запись удалена.
  При удалении мы просто меняем один этот байт на `1` (помечаем как «удаленная»). Программа при чтении просто игнорирует такие записи, а файл не нужно перезаписывать!

29. Что такое дефрагментация файловой базы?

Это процесс физической чистки базы данных после множества логических удалений (из вопроса 28).

Программа создает новый чистый файл, копирует туда только **активные** записи (выкидывая все помеченные как удаленные) и заменяет старый файл новым. В результате удаляются «дыры», а размер файла на диске уменьшается.

30. В каких случаях целесообразно использовать memory-mapped files?

1. **При обработке очень больших файлов** (видео, базы данных, дампы), которые не помещаются целиком в оперативную память (через скользящие окна, как в нашей лабе).
2. **При произвольном (случайном) доступе к данным**, когда нужно постоянно прыгать по разным кускам файла (намного удобнее указателей `ptr[i]`, чем `ReadFile`).
3. **Для разделяемой памяти между процессами (Inter-Process Communication, IPC):** два разных процесса могут отобразить один и тот же файл и обмениваться данными через память с безумной скоростью.
4. **При частом чтении одних и тех же структур данных**, чтобы избежать накладных расходов на постоянное копирование буферов.